

5-4 · Spark DataFrame과 SparkSQL 데이터 가공 · 실습 1H

SparkSQL로 집계와 뷰 만들기

같은 가공을 SQL 로 — 계획을 대조하고 Iceberg 에 저장

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 임시 뷰 `orders_v`·`products_v` 를 등록하고 `SHOW VIEWS` 로 확인한다
- ② 같은 집계를 SQL 로 써서 API 결과와 완전히 같은지 대조한다
- ③ 두 `explain()` 의 Physical Plan 이 같은 구조임을 눈으로 확인한다
- ④ CTE 로 단계를 나누고 결과를 Iceberg 테이블로 저장한다

앞 챕터의 셀과 `orders` · `products` · `sales` 변수를 그대로 이어서 씁니다

여섯 단계 — 뷰 등록에서 미니 과제까지

임시 뷰는 이 셀(세션)이 살아 있는 동안만 존재합니다 — 셀을 나가면 사라집니다

1. 임시 뷰	2. SQL 집계	3. 계획 대조	4·5. CTE·저장	6. 미니 과제
orders_v products_v	API 결과와 대조	explain() 둘 나란히	누적 매출 Iceberg	left_anti 비중 %

뷰를 등록하고 같은 집계를 SQL 로

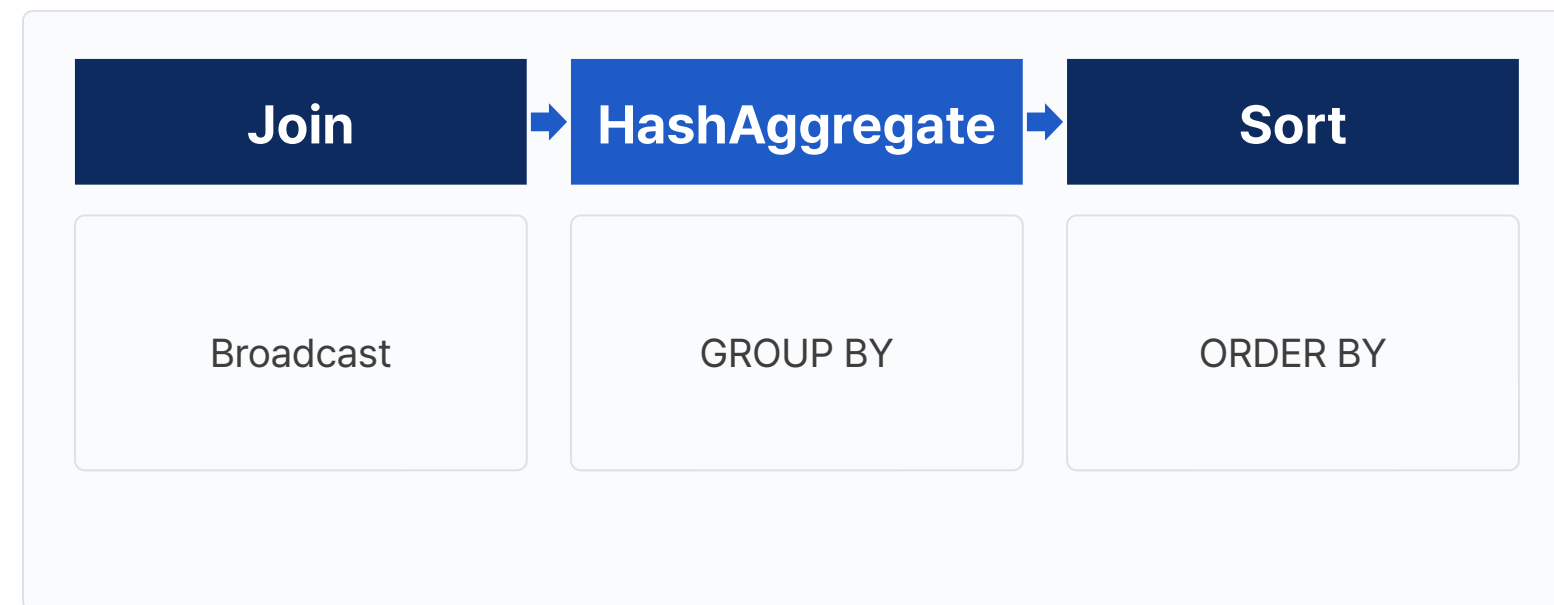
뷰 등록	orders.createOrReplaceTempView("orders_v")
확인	spark.sql("SHOW VIEWS").show()
SELECT	p.name, SUM(o.qty * p.price) AS total_amount, SUM(o.qty) AS total_qty
FROM	orders_v o JOIN products_v p ON o.product_id = p.product_id
GROUP BY	p.name ORDER BY total_amount DESC

SQL 결과가 앞 챕터의 API 결과와 완전히 같아야 한다 — 값이 다르면 조인 조건이나 집계 대상을 다시

두 explain() 을 나란히 — 같은 엔진의 증거

sql_sales.explain()

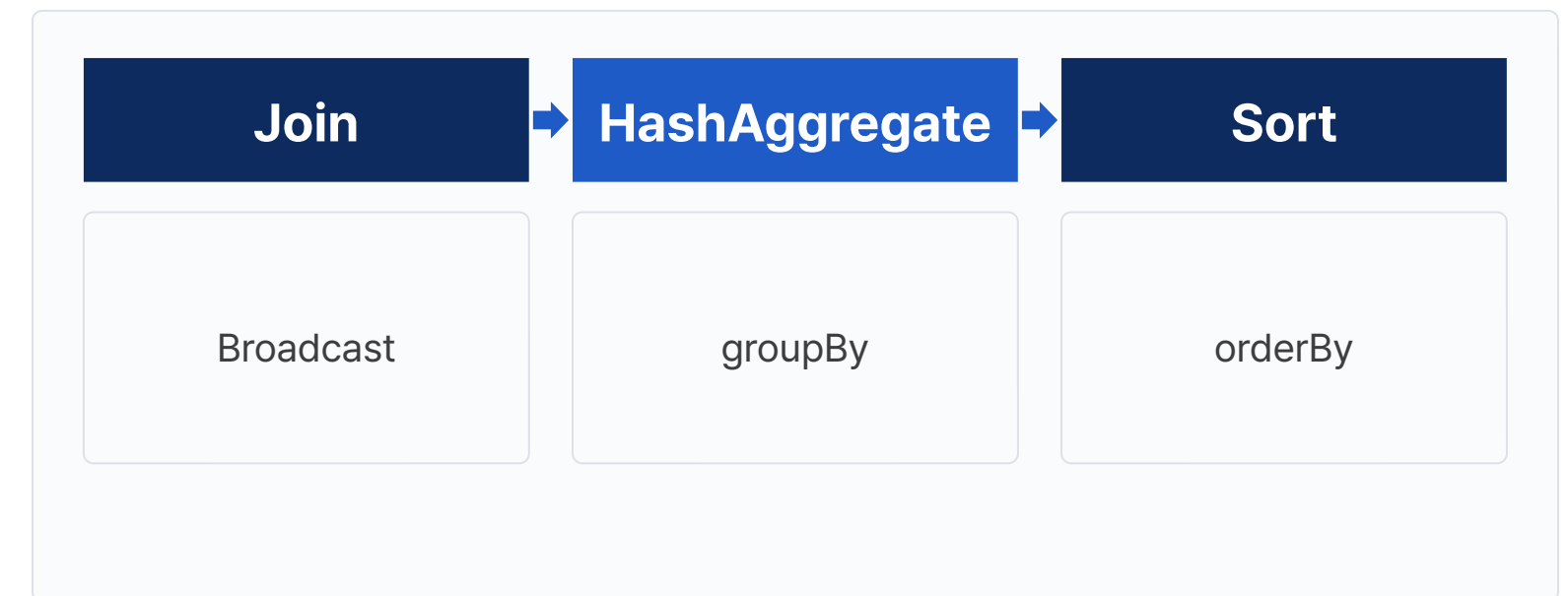
SQL 문장에서 나온 Physical Plan



VS

sales.explain()

컬럼 순서·별칭은 달라도 연산 종류·순서가 같으면 같은 계획



4단계

CTE 로 단계를 나눈다 — 2-2 문법 그대로



2-2 · 3-4 에서 쓴 것과 같은 문법 — 긴 쿼리를 읽기 좋게 나눈다

WITH enriched AS (...), daily AS (...) SELECT ... — SQL 을 익혀 두면 도구가 바뀌어도 그대로 쓰인다

결과를 Iceberg 테이블로 저장 — 방법 넷

방법	한 줄	동작
<code>createOrReplace()</code>	<code>sales.writeTo("lake.db.product_sales") .createOrReplace()</code>	없으면 만들고 있으면 통째로 교체
<code>append()</code>	<code>.writeTo(...).append()</code>	뒤에 붙임
<code>overwritePartitions()</code>	<code>.writeTo(...).overwritePartitions()</code>	해당 파티션만 교체
SQL CTAS	<code>CREATE OR REPLACE TABLE lake.db.daily_revenue USING iceberg AS SELECT ...</code>	4-4 의 Trino CTAS 와 같은 형태

저장 뒤 `SELECT * FROM lake.db.product_sales` 가 `sales` 와 같고, MinIO 의 `warehouse/db/product_sales/` 아래 파일이 생긴다

show() 를 빼고 실행하면 — 아무것도 없다

이론의 「지연 평가(action 을 불러야 실행)」를 직접 겪어 봅니다

1. show() 없이

q = spark.sql(...)
만 실행

2. 아무것도 없다

transformation
뿐 — 지연 평가

3. q.show()

action 을
불러야 실행

4. ... 으로 잘림

기본 20행
truncate=True

5. 처방

show(
truncate=False)

미니 과제 셋 — 5-5 데이터 마트의 뼈대

1 날짜별 매출을 API 와 SQL 로 각각
결과를 대조 — 5-5 에서 만들 데이터 마트의 뼈대가 된다

2 주문이 한 건도 없는 상품
`products.join(orders, on="product_id", how="left_anti")` — 이 데이터에선 0행

3 상품별 매출 비중(%)
`total_amount * 100.0 / SUM(total_amount) OVER ()` — 합이 100 에 가까워야

`left_anti`

**0행이면
정답**

이 1시간이 끝날 때 확인할 것

- 1 SHOW VIEWS 에 orders_v · products_v 가 보인다
- 2 SQL 집계 결과가 API 의 sales 와 완전히 같다
- 3 두 explain() 이 Join → HashAggregate → Sort 같은 구조다
- 4 CTE 쿼리에 날짜별 매출과 누적 매출이 나온다
- 5 lake.db 의 product_sales · daily_revenue 가 조회되고 MinIO 에 파일이 있다
- 6 left_anti 결과 0행 · 비중의 합이 100 에 가깝다

5-4 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

Pandas · PySpark · SQL 세 도구를 한 표로 나란히 놓고, 5-5 파티셔닝을 위한 준비를 확인합니다

✓ 세 도구 대조표 한 장

✓ SHOW TABLES IN lake.db 로 테이블 세 개

✓ 셔플 파티션 200 확인

✓ 자주 겪는 오류 네 가지

가져갈 것

셀 그대로 · sales 변수 살아 있게